

[https://moodle.joinville.udesc.br/pluginfile.php/428990/mod\\_resource/content/3/PAA\\_02.pdf](https://moodle.joinville.udesc.br/pluginfile.php/428990/mod_resource/content/3/PAA_02.pdf)

# Projeto e Análise de Algoritmos

Medida de complexidade, notação  $O$

André Tavares da Silva  
andre.silva@udesc.br

# Funções de Complexidade

---

Considere que cada operação leva 1ns em média em um determinado processador. Determine o tempo das funções abaixo para os seguintes valores de operações:

| $f(n) / n$   | $n=10$ | $n=100$ | $n=1.000$ | $n=10.000$ | $n=100.000$ | $n=1.000.000$ |
|--------------|--------|---------|-----------|------------|-------------|---------------|
| $\log_2 n$   |        |         |           |            |             |               |
| $n$          |        |         |           |            |             |               |
| $3n$         |        |         |           |            |             |               |
| $n \log_2 n$ |        |         |           |            |             |               |
| $n^2$        |        |         |           |            |             |               |
| $2^n$        |        |         |           |            |             |               |
| $n!$         |        |         |           |            |             |               |

# Funções de Complexidade

Considere que cada operação leva 1ns em média em um determinado processador. Determine o tempo das funções abaixo para os seguintes valores de operações:

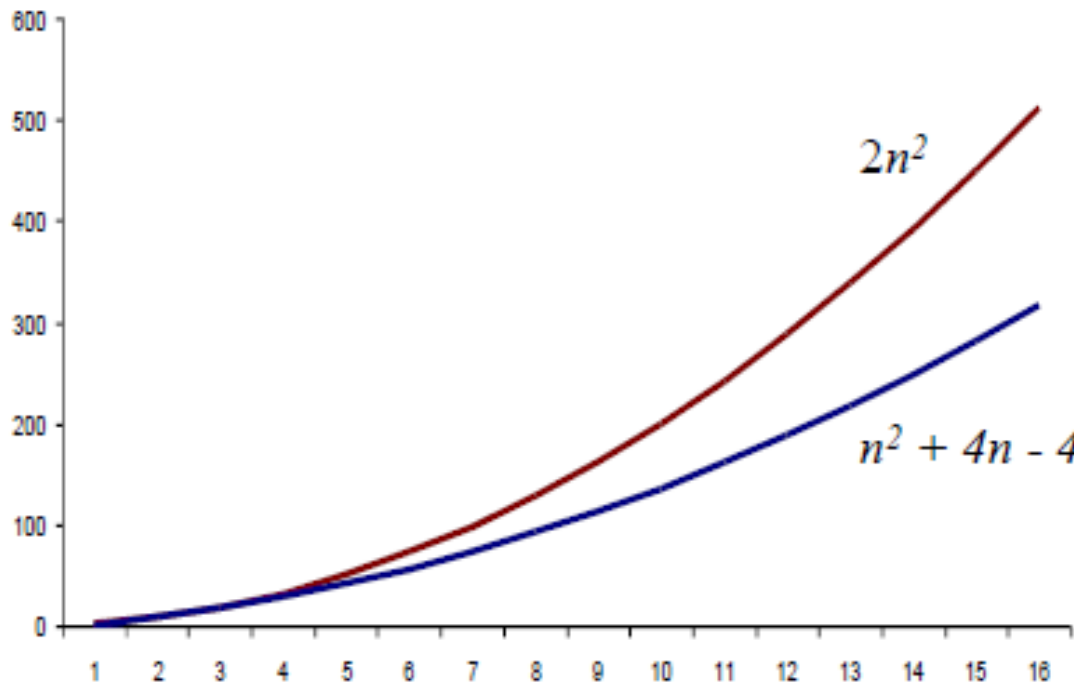
| $f(n) / n$   | $n=10$  | $n=100$     | $n=1.000$    | $n=10.000$    | $n=100.000$    | $n=1.000.000$  |
|--------------|---------|-------------|--------------|---------------|----------------|----------------|
| $\log_2 n$   | 3,3     | 6,6         | 10,0         | 13,3          | 16,6           | 19,9           |
| $n$          | 10      | 100         | 1000         | 10000         | 100000         | 1000000        |
| $3n$         | 30      | 300         | 3000         | 30000         | 300000         | 3000000        |
| $n \log_2 n$ | 33,2    | 664,4       | 9.965,8      | 132.877,1     | 1.660.964,0    | 19.931.568,6   |
| $n^2$        | 100     | 10000       | 1000000      | 100000000     | 10000000000    | 1000000000000  |
| $2^n$        | 1024    | 1,2676E+030 | 1,0715E+301  | 1,995E+3.010  | 1,0E+30.103    | 1,0E+301.030   |
| $n!$         | 3628800 | 9,3326E+157 | 4,023E+2.567 | 2,846E+33.659 | 2,824E+456.573 | 1,0E+5.565.709 |

# Notação Assintótica

## (Notação O grande – Limite Superior)

---

Uma função  $g(n)$  domina assintoticamente outra função  $f(n)$  se existem duas constantes positivas  $c$  e  $n_0$  tais que, para  $n > n_0$ , temos  $|f(n)| \leq c \cdot |g(n)| \rightarrow f(n) = O(g(n))$



$$n^2 + 4n - 4 = O(n^2)$$

$$2n^2 = O(n^2)$$

# Notação Assintótica

## (Notação O grande – Limite Superior)

---

- Ao invés de nos preocuparmos em contar exatamente o número de instruções de um programa, é mais tratável matematicamente analisar a ordem de magnitude da função  $T(n)$ , ou seja, o que acontece com a função  $T(n)$  quando  $n$  cresce arbitrariamente.
- Suponha de exista uma função  $f(n)$ , definida para todos os inteiros maiores ou iguais a zero, tal que para constantes  $c$ , temos:  $T(n) \leq c \cdot f(n)$ .
- Para  $n$  suficientemente grande, dizemos que  $T(n) = O(f(n))$ .

# Notação Assintótica

## (Notação O grande – Limite Superior)

---

- Considere o código abaixo:

```
int sum_of_matrix(M, n) {  
    soma = 0;  
    for(i=0;i<n;i++) {  
        soma_linha = 0;  
        for(j=0;j<n;j++)  
            soma_linha += M[i,j];  
        soma += soma_linha;  
    }  
    return soma;  
}
```

# Notação Assintótica

## (Notação O grande – Limite Superior)

---

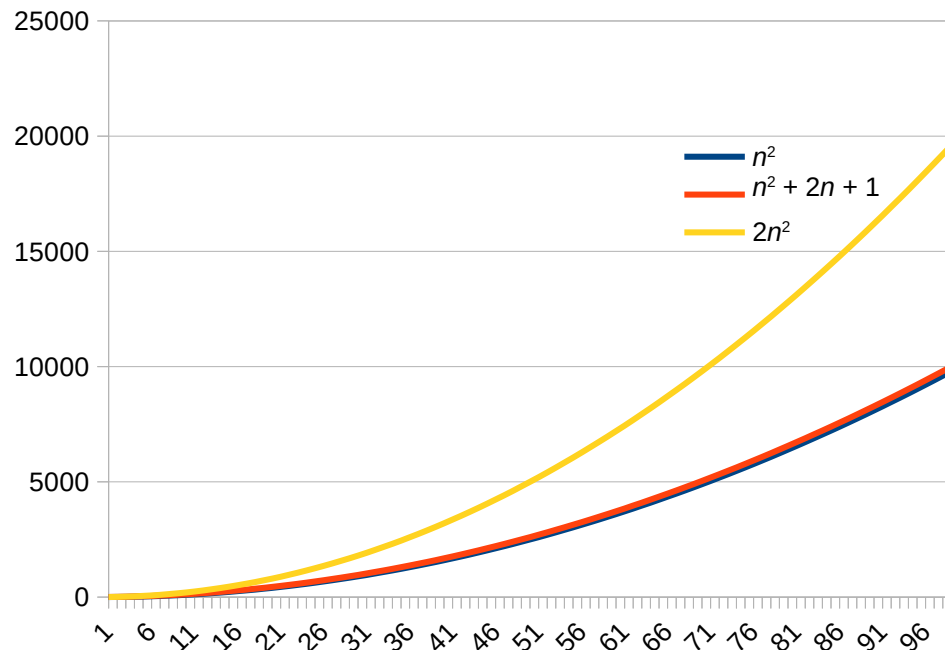
```
int sum_of_matrix(M, n) {  
    soma = 0;  
    for(i=0;i<n;i++) {  
        soma_linha = 0;  
        for(j=0;j<n;j++)  
            soma_linha += M[i,j];  
        soma += soma_linha;  
    }  
    return soma;  
}
```

- Calculando o número de instruções: no laço mais interno temos  $n$  iterações, enquanto no mais externo temos  $n+2$  atribuições. Chegamos a  $T(n) = n(n+2)+1 = \mathbf{n^2 + 2n + 1}$

# Notação Assintótica

## (Notação O grande – Limite Superior)

- Calculando o número de instruções: no laço mais interno temos  $n$  iterações, enquanto no mais externo temos  $n+2$  atribuições.  
Chegamos a  $T(n) = n(n+2)+1 = \mathbf{n^2 + 2n + 1}$
- Note que, se tomarmos  $c=2$  e  $f(n) = n^2$ , temos:  
 $2n^2 > n^2 + 2n + 1$  para todo  $n > 2$





# Hierarquia de funções

---

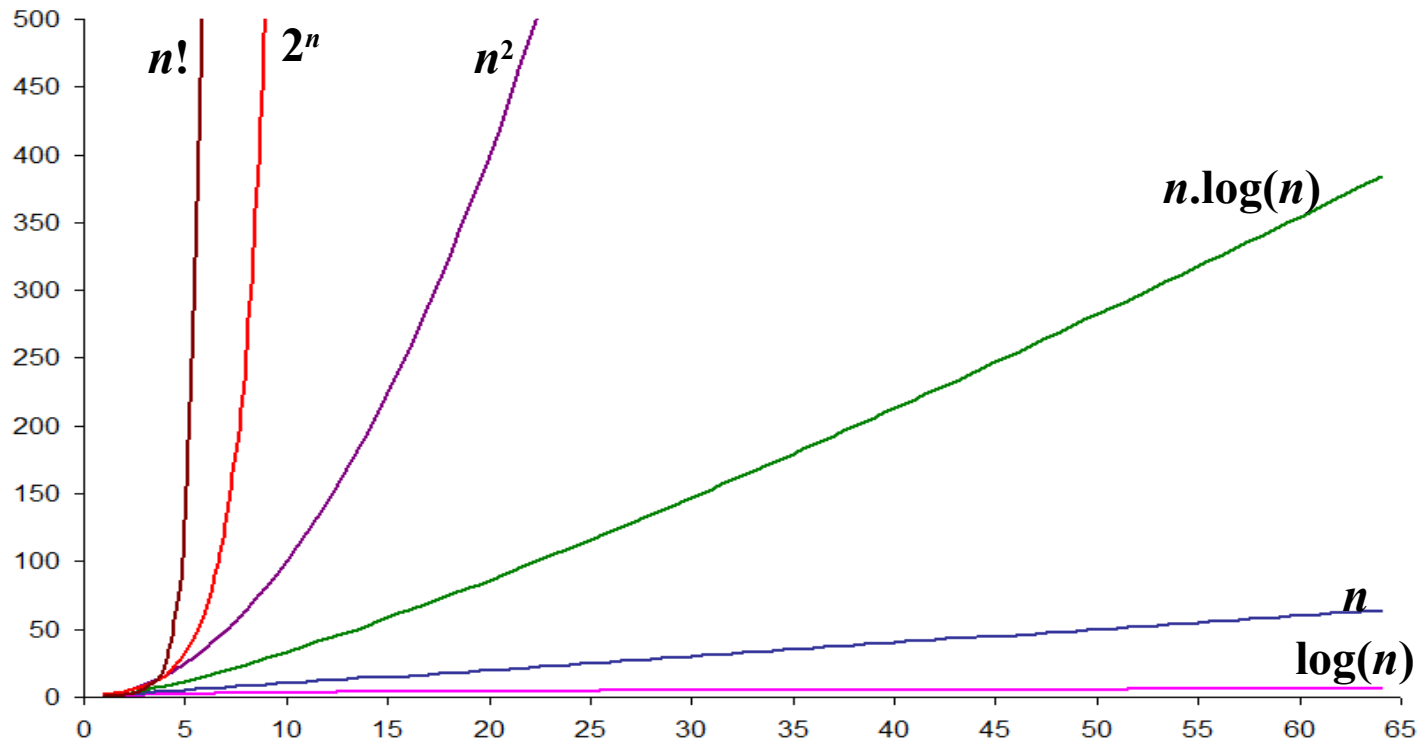
Hierarquia de funções do ponto de vista assintótico:

$$1 < \log \log n < \log n < n^\varepsilon < n^c < n^{\log n} < c^n$$

onde  $\varepsilon$  e  $c$  são constantes arbitrárias tais que  $0 < \varepsilon < 1 \leq c$ .

# Hierarquia de funções

**CONSIDERAÇÃO I:** trabalhar com valores grandes para ‘n’ (entrada). Assim, ordens de crescimento são destacadas.



- Considere o código abaixo:

```
sum_of_n(n) {  
    soma = 0  
    for i = 1 to n  
        soma = soma + i  
    return soma  
}
```

- Considere o código abaixo:

```
sum_of_n(n) {  
    soma = 0  
    for i = 1 to n  
        soma = soma + i  
    return soma  
}
```

A instrução de inicialização é executada apenas uma vez e que o laço ***for*** executa  $n$  vezes, o que nos leva a:  $T(n) = n + 1$

- Considere o código abaixo:

```
Func_A(n) {  
    c = 0  
    for i = 1 to n  
        c += 1  
    for j = 1 to n:  
        c += 2  
    return c  
}
```

- Considere o código abaixo:

```
Func_A(n) {  
    c = 0  
    for i = 1 to n  
        c += 1  
    for j = 1 to n:  
        c += 2  
    return c  
}
```

A inicialização é executada uma vez e logo em seguida temos dois laços *for* com  $n$  iterações, o que nos leva a:  $T(n) = 2n + 1$

# Medidas de Complexidade

---

**CONSIDERAÇÃO II:** Ignorar o custo das instruções (tempo constante) e focar na análise do crescimento do uso de um recurso (tempo, espaço) em relação ao crescimento da entrada.

**Ex:** ordenar uma lista de ‘n’ elementos e mostrar a lista ordenada

| <i>n</i> | Ordenação Bolha | printf vetor |
|----------|-----------------|--------------|
| 100      | 37,8 $\mu$ s    | 8,532 ms     |
| 200      | 148,4 $\mu$ s   | 17,847 ms    |
| 1.000    | 3,748 ms        | 91,569 ms    |
| 10.000   | 247 ms          | 860,205 ms   |
| 50.000   | 5,307 s         | 4,277 s      |
| 100.000  | 20,422 s        | 8,693 s      |

# Medidas de Complexidade

---

**CONSIDERAÇÃO III:** pode-se analisar os valores de entrada com perspectivas diferentes:

- **Melhor caso**  $\Rightarrow$  menor complexidade para um valor de 'n';
- **Pior caso**  $\Rightarrow$  maior complexidade para um valor de 'n';
- **Complexidade esperada ou média**  $\Rightarrow$  leva-se em conta a probabilidade de ocorrência de cada entrada de um mesmo tamanho 'n'.

Pode-se antecipar alguma relação ( $<$ ,  $\leq$ ,  $>$ ,  $\geq$ ) entre as complexidades média e pior caso de um algoritmo qualquer?



# Medidas de Complexidade

---

```
int pesquisa(Estrutura *v, int n, int chave) {  
    int i;  
    for (i = 0; i < n; i++)  
        if (v[i].chave == chave)  
            return i;  
    return -1;  
}
```

Em que situação ocorre o melhor caso?

Em que situação ocorre o pior caso? E o caso médio?

**ATENÇÃO:** não assuma um valor fixo e pequeno para  $n$  ao considerar o melhor caso!

# Medidas de Complexidade

---

**Melhor caso:** Caso o primeiro registro seja o registro procurado será necessária apenas uma comparação.

Logo, podemos dizer que a complexidade é **constante**

[OBS: existe uma notação especial para indicar melhor caso – veremos ela mais adiante]

# Medidas de Complexidade

---

**Pior caso:** Caso o último registro acessado seja aquele que se procura:

```
int pesquisa(Estrutura *v, int n, int chave) {  
    int i;                                // O(1)  
    for (i = 0; i < n; i++)               // O(n)  
        if (v[i].chave == chave)         // O(1)  
            return i;                     // O(1)  
    return -1;                             // O(1)  
}
```

Logo, podemos dizer que a função pesquisa tem complexidade **O(n)** para o pior caso.

# Medidas de Complexidade

---

**Caso médio:** Caso o *i-ésimo* registro seja o registro procurado são necessárias *i* comparações. Sendo  $p_i$  a probabilidade de procurarmos o *i-ésimo* registro temos:

$$f(n) = 1.p_1 + 2.p_2 + \dots + n.p_n.$$

Considerando que a probabilidade de procurar qualquer registro é a mesma probabilidade, temos:

$$p_i = 1 / n \quad \text{para todo } i.$$

$$f(n) = \frac{1}{n} (1 + 2 + \dots + n) = \frac{1}{n} \left( \frac{n(n+1)}{2} \right) = \frac{(n+1)}{2}$$

Logo, temos uma complexidade linear:  $\frac{(n+1)}{2} = (n)$

# Medidas de Complexidade

---

**CONSIDERAÇÃO IV:** pode-se analisar a complexidade em relação a diferentes recursos. Os mais usuais são: tempo e espaço.

Complexidade de espaço:

Devemos considerar todo o espaço adicional criado pelo algoritmo assim como a quantidade de chamadas de função (geralmente recursivas) realizadas.

Por enquanto vamos focar no recurso “tempo”!

# Exemplo (Bubble Sort)

---

```
void bubble(int *v, int n){  
    int i, j, aux;  
    for (i = n - 1; i > 0; i--){  
        for (j = 0; j < i; j++){  
            if (v[j] > v[j+1]){  
                aux = v[j];  
                v[j] = v[j+1];  
                v[j+1] = aux;  
            }  
        }  
    }  
}
```

Deveria ter uma flag  
senão teve alguma troca  
pode parar.

# Análise de Complexidade (Bubble Sort)

---

Iremos considerar 3 cenários distintos: pior caso, caso médio e melhor caso.

**Pior caso:** vetor em ordem decrescente.

Observando a função definida anteriormente, note que no pior caso o segundo loop vai de 1 a  $i$ , sendo que na primeira vez  $i = n-1$ , na segunda vez  $i = n-2$  e até  $i = 1$ . Sendo assim, o número de operações é dado por:

$$f(n) = (n-1) + (n-2) + \dots + 1$$

# Análise de Complexidade (Bubble Sort)

---

O número de operações é dado por:

$$f(n) = (n-1) + (n-2) + \dots + 1$$

Como já vimos, o somatório  $1 + 2 + \dots + n$  é igual a  $n(n+1)/2$ .  
Assim, temos:

$$f(n) = \frac{n(n-1)}{2} = \frac{n^2 - n}{2}$$

O que nos leva a  **$O(n^2)$** .



# Análise de Complexidade (Bubble Sort)

---

**Melhor caso:** vetor em ordem crescente.

Se uma lista já está ordenada e o Bubblesort não realiza nenhuma troca, o algoritmo pode terminar após o primeiro passo. Podemos fazer uma modificação no algoritmo original contando quantas trocas foram realizadas. Com essa modificação, se o algoritmo encontra uma lista ordenada, sua complexidade é  **$O(n)$** , pois ele percorre a lista de  $n$  elementos uma única vez:  $f(n) = n - 1$

# Análise de Complexidade (Bubble Sort)

---

**Caso médio:** devemos tirar uma média de todos os possíveis casos, supondo que são equiprováveis.

Lembre-se que na primeira iteração temos  $n-1$  trocas, na segunda iteração temos  $n-2$  e assim sucessivamente, até atingirmos 1 única troca na última iteração. Portanto, é como se tirássemos uma média do somatório do caso anterior para diversos valores de  $k$ , variando de 1 até  $n-1$ .

$$f(n) = \frac{1}{n-1} \sum_{k=1}^{n-1} \left( \sum_{i=1}^k i \right) = \dots = \frac{1}{6} (n^2 + n)$$

O que nos leva a  **$O(n^2)$** .

# Exemplo

## (Ordenação por Inserção)

---

```
void insercao(int *v, int n){  
    int i, j, x;  
    for (i = 1; i < n; i++){  
        x = v[i];  
        j = i - 1;  
        while (j >= 0 && v[j] > x){  
            v[j+1] = v[j];  
            j--;  
        }  
        v[j+1] = x;  
    }  
}
```

# Exemplo

## (Ordenação por Inserção)

---

Insertion Sort, ou ordenação por inserção, é o algoritmo de ordenação que, dado um vetor inicial constrói um vetor final com um elemento de cada vez, uma inserção por vez. Como também é um algoritmo quadrático, é eficiente para poucas entradas.

O funcionamento é semelhante ao inserir as cartas de um baralho uma a uma, colocando sempre na posição correta. Primeiramente busca-se qual a posição correta até encontrar uma carta maior, sendo adicionada na posição correta (e ajustando as demais - *shift*).

# Exemplo

## (Ordenação por Inserção)

---

Exemplo:

[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]

1a passagem: [5, **2**, 13, 7, -3, 4, 15, 10, 1, 6] → [2, 5, 13, 7, -3, 4, 15, 10, 1, 6]

2a passagem: [2, 5, **13**, 7, -3, 4, 15, 10, 1, 6] → [2, 5, 13, 7, -3, 4, 15, 10, 1, 6]

3a passagem: [2, 5, 13, **7**, -3, 4, 15, 10, 1, 6] → [2, 5, 7, 13, -3, 4, 15, 10, 1, 6]

4a passagem: [2, 5, 7, 13, **-3**, 4, 15, 10, 1, 6] → [-3, 2, 5, 7, 13, 4, 15, 10, 1, 6]

5a passagem: [-3, 2, 5, 7, 13, **4**, 15, 10, 1, 6] → [-3, 2, 4, 5, 7, 13, 15, 10, 1, 6]

6a passagem: [-3, 2, 4, 5, 7, 13, **15**, 10, 1, 6] → [-3, 2, 4, 5, 7, 13, 15, 10, 1, 6]

7a passagem: [-3, 2, 4, 5, 7, 13, 15, **10**, 1, 6] → [-3, 2, 4, 5, 7, 10, 13, 15, 1, 6]

8a passagem: [-3, 2, 4, 5, 7, 10, 13, 15, **1**, 6] → [-3, 1, 2, 4, 5, 7, 10, 13, 15, 6]

9a passagem: [-3, 1, 2, 4, 5, 7, 10, 13, 15, **6**] → [-3, 1, 2, 4, 5, 6, 7, 10, 13, 15]

# Análise de Complexidade (Ordenação por Inserção)

---

**Melhor caso:** note que quando o array já está ordenado, os pivôs já estão nas posições corretas. Assim, nenhuma troca será necessária. Logo:

$$f(n) = \sum_{i=2}^n 1 = n - 1$$

O que resulta em  **$O(n)$** .

# Análise de Complexidade (Ordenação por Inserção)

---

**Pior caso:** note que neste caso, o primeiro pivô realizará 1 troca, o segundo pivô realizará 2 trocas, e assim sucessivamente, de modo que o segundo loop vai ter de percorrer todo vetor ( $i-1$  trocas).

$$f(n) = n^2 - 1$$

O que resulta em  $O(n^2)$ .

# Análise de Complexidade (Ordenação por Inserção)

---

**Caso médio:** podemos aplicar uma estratégia muito similar àquela adotada na análise do Bubblesort. A ideia consiste em considerar que todos os casos são igualmente prováveis e calcular uma média de todos eles. Assim, temos:

$$f(n) = \frac{1}{3}n^2 + \frac{5}{6}n$$

O que resulta em  $O(n^2)$ .



# Exemplo

## (Ordenação por Seleção)

---

```
void selectionSort(int *v, int n){  
    int i, j, x, aux;  
    for (i = 0; i < n; i++){  
        x = i;  
        for (j = i+1; j < n; j++){  
            if( v[j] < v[x] )  
                x = j;  
        }  
        aux = v[i];  
        v[i] = v[x];  
        v[x] = aux;  
    }  
}
```

# Exemplo

## (Ordenação por Seleção)

---

A ordenação por seleção é um método baseado em passar o menor valor do vetor para a primeira posição mais a esquerda disponível, depois o de segundo menor valor para a segunda posição a esquerda e assim sucessivamente, com os  $n - 1$  elementos restantes.

Esse algoritmo compara a cada iteração um elemento com os demais, visando encontrar o menor. A complexidade desse algoritmo será sempre de ordem quadrática, isto é o número de operações realizadas depende do quadrado do tamanho do vetor de entrada.

# Exemplo

## (Ordenação por Seleção)

---

Exemplo:

[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]

1a passagem: [5, 2, 13, 7, **-3**, 4, 15, 10, 1, 6] → [**-3**, 2, 13, 7, 5, 4, 15, 10, 1, 6]

2a passagem: [-3, 2, 13, 7, 5, 4, 15, 10, **1**, 6] → [-3, **1**, 13, 7, 5, 4, 15, 10, 2, 6]

3a passagem: [-3, 1, 13, 7, 5, 4, 15, 10, 2, 6] → [-3, 1, **2**, 7, 5, 4, 15, 10, 13, 6]

4a passagem: [-3, 1, 2, 7, 5, **4**, 15, 10, 13, 6] → [-3, 1, 2, **4**, 5, 7, 15, 10, 13, 6]

5a passagem: [-3, 1, 2, 4, **5**, 7, 15, 10, 13, 6] → [-3, 1, 2, 4, **5**, 7, 15, 10, 13, 6]

6a passagem: [-3, 1, 2, 4, 5, 7, 15, 10, 13, **6**] → [-3, 1, 2, 4, 5, **6**, 15, 10, 13, 7]

7a passagem: [-3, 1, 2, 4, 5, 6, 15, 10, 13, 7] → [-3, 1, 2, 4, 5, 6, **7**, 10, 13, 15]

8a passagem: [-3, 1, 2, 4, 5, 6, 7, **10**, 13, 15] → [-3, 1, 2, 4, 5, 6, 7, **10**, 13, 15]

9a passagem: [-3, 1, 2, 4, 5, 6, 7, 10, **13**, 15] → [-3, 1, 2, 4, 5, 6, 7, 10, **13**, 15]

# Análise de Complexidade (Ordenação por Seleção)

---

**Pior caso:** note que no pior caso o segundo loop vai de  $i+1$  até  $n$ , sendo que na primeira vez  $i = 1$ , na segunda vez  $i = 2$  e até  $i = n - 1$  (a atribuição dentro do *for* é realizada toda vez). Sendo assim, o número de operações é dado por:

$$f(n) = \frac{1}{2}n^2 + \frac{3}{2}n$$

O que resulta em  $O(n^2)$ .

# Análise de Complexidade (Ordenação por Seleção)

---

**Melhor caso:** note que mesmo que a lista já esteja ordenada, o loop *for* interno será executado todas as vezes. Uma desvantagem do algoritmo Selectionsort é que mesmo no melhor caso, para encontrar o menor elemento, devemos percorrer todo o restante do vetor no loop mais interno.

Assim o melhor caso também é  $O(n^2)$ .

**Caso médio:** como tanto melhor como pior caso são quadráticos, o caso médio também é  $O(n^2)$ .

# Atividade

---

- Implemente as funções de ordenação analisadas e faça um quadro comparativo do tempo de execução para ordenar:
  - (n=)100.000 números aleatórios  
(OBS: utilize a mesma entrada para cada algoritmo)
  - (n=)100.000 números já em ordem crescente  
(OBS: pode ser uma sequencia simples como 1,2,3, ..., 100.000)

# Atividade

---

Elabore os seguintes algoritmos e analise o seu tempo de execução para diferentes entradas e determine a sua complexidade de tempo.

- Implemente um algoritmo (função) que recebe como parâmetro dois valores inteiros  $a$  e  $b$  e calcula  $a^b$ .
- Implemente um algoritmo (função) que recebe duas matrizes quadradas de mesma ordem ( $n \times n$ ) e realiza a multiplicação entre elas.